Computational Capacity, Architecture, and Decision Requirements in the Evolution of Decision Support Systems: A Critical Narrative Review

Unattributed review manuscript

***Abstract*—Decision support systems (DSS) have developed through overlapping model-based, knowledge-oriented, collaborative, data-intensive, and increasingly language-mediated approaches. Explaining these developments requires distinguishing the resources that make a computation feasible from the representations and organizational arrangements that make its results useful. This critical narrative review examines recurring constraints in decision support and compares the contributions of computational capacity, architecture, data and expertise, and decision requirements. Historical DSS frameworks are considered alongside technical contributions on distributed execution, learned representations, retrieval, and tool use. The synthesis identifies a recurring pattern: a response that relaxes one constraint often leaves or relocates another, including integration, knowledge maintenance, interpretation, coordination, and action validation. Reported MapReduce and Transformer experiments illustrate interactions between architecture and available resources; their benchmark results do not establish causes of organizational DSS adoption. A comparison of decision-support configurations and a proposed diagnostic framework distinguish situations that call for scaling, architectural redesign, improved evidence, or revised decision formulation. The review's contribution is an explicit comparison of these mechanisms, rather than a universal sequence of generations or a causal estimate of historical change.**

***Index Terms*—decision support systems, computational capacity, architecture, narrative review, decision requirements, business intelligence, large language models**

I. Introduction

Decision support systems assist people in understanding situations, evaluating alternatives, and selecting or implementing actions. Their contribution can take different forms: an optimization model compares feasible choices; a knowledge base applies domain expertise; an analytical platform integrates evidence; and a predictive model estimates outcomes. These functions should be distinguished because improved computation does not automatically improve the decision in which its output is used.

Historical accounts document the expansion from individual analytical support toward group, organizational, and Web-based applications [1], [2]. Primary frameworks also show that technical resources and decision formulation have long interacted. Little emphasizes models that managers can understand, control, and adapt; Gorry and Scott Morton distinguish the information needed for different managerial activities [3], [4]. These arguments caution against explaining DSS evolution solely through faster hardware or larger databases.

Computational capacity affects feasible workload size, memory use, latency, and operating cost. Architecture determines how data, representations, models, interfaces, and actions are connected. Data and expertise supply the material on which the system operates. Decision requirements specify the objectives, constraints, preferences, participation, and accountability that make a result useful. The four dimensions interact, but they are not interchangeable. More processing power can accelerate an unsuitable model, while a useful representation may remain impractical until adequate resources become available.

This review asks three questions: RQ1, which recurring constraints have DSS approaches addressed? RQ2, how do capacity and architecture contribute differently to those responses? RQ3, which unresolved decision requirements remain after a technical improvement? The contribution is a consistent comparison of constraints and responses. It does not assume that every organization passed through the same stages or that newer components displace earlier methods.

II. Review Approach and Relationship to Prior Work

A. Scope and Source Use

The review uses a purposive set of 27 works combining foundational DSS frameworks, historical and contemporary reviews, decision-modeling literature, and technical mechanism papers. Sources were selected for their relevance to four extraction questions: the decision-support problem, the capacity constraint, the architectural response, and the remaining limitation. The companion source register records the role and assessed version of each reference. Coverage is selective and access-dependent; it is not a representative sample of the entire DSS literature.

The synthesis distinguishes three uses of evidence. Historical frameworks and reviews support descriptions of DSS purposes and configurations. Technical studies support claims about the mechanisms and evaluated tasks they report. Connections between those mechanisms and organizational design choices are interpretive synthesis unless the cited work directly investigates the organization-level relationship. A benchmark improvement is therefore evidence of a tested capability, not by itself evidence of widespread adoption or improved managerial decisions.

No systematic database search, meta-analysis, new experiment, or quantitative causal reconstruction is reported. The

review does not establish an exhaustive state of the art through 2026. This scope permits close comparison of illustrative mechanisms while limiting claims about prevalence, dates of adoption, and the relative historical importance of hardware, algorithms, or organizational change.

B. Analytical Units and Comparison Dimensions

The main comparison unit is a decision-support configuration: a combination of decision purpose, information representation, processing components, and user participation. The configurations overlap. For example, a knowledge-oriented system can use a Web interface, and a collaborative system can incorporate predictive models. Power's distinction between dominant technology, users, purpose, and deployment helps prevent these dimensions from being treated as equivalent generations [2].

Four dimensions guide the comparison. *Capacity* concerns resources required to execute a suitable workload at acceptable scale, latency, and cost. *Architecture* concerns the organization of representations, components, interfaces, and control flow. *Data and expertise* concern availability, quality, provenance, and maintenance. *Decision requirements* concern what should be optimized, which constraints apply, who participates, and who is authorized to act. The categories are analytical distinctions, not independently measured causes.

C. Positioning Against Existing Reviews

The broad development of DSS is established prior work. Shim *et al.* review historical tools, collaboration, and deployment; Power provides a classification rather than a replacement chronology [1], [2]. More recent reviews examine architectural shifts toward strategic AI and combine DSS taxonomy, applications, organizational context, and governance [5], [6]. These works already discuss hybrid systems, feedback, and continuing human involvement. The present review does not claim those observations as new discoveries.

Its narrower emphasis is the distinction between computational feasibility and decision adequacy: what a larger resource budget enables, what a changed representation or interface enables, and what neither resolves. The related reviews provide context for that comparison, while the original technical and DSS sources support the specific mechanisms discussed below. The resulting framework is an interpretive synthesis, not a separately validated theory of historical causation.

III. Recurring Requirements and DSS Configurations

A. Interactive Model-Based Support

Interactive model-based support responds to the limitation of fixed reporting when a decision maker needs to change assumptions, compare alternatives, or inspect sensitivity. Early DSS frameworks associate useful support with a suitable model, relevant data, and a workable dialogue between the system and its user [1], [3], [4]. Short-turnaround calculation helps make repeated exploration practical, but usability depends on how the problem is represented and how the manager can interrogate it.

Little's decision-calculus argument is especially relevant: model detail can make a system harder to understand and control, even when the computation is technically feasible [3]. Gorry and Scott Morton likewise argue that strategic information requirements differ in character from operational requirements; managerial support need not depend on ever-larger aggregates of operational detail [4]. These are primary conceptual and design arguments, rather than controlled comparisons of organizational outcomes.

The capacity contribution is faster or more extensive calculation. The architectural contribution is an accessible model–data–dialogue arrangement. Persistent limitations include inappropriate assumptions, difficult calibration, and objectives that do not reflect the decision context. Improving numerical throughput does not resolve those limitations without changes to representation or decision formulation.

B. Knowledge-Oriented Support

Knowledge-oriented support addresses tasks in which domain expertise is difficult to express as a single numerical objective or is unavailable to every decision maker. Feigenbaum describes the acquisition, representation, and use of expert knowledge, including explanatory reasoning [7]. Later expert-system literature documents multiple methods and applications rather than a uniform architecture or universal successor to analytical models [8].

Additional memory and processing can accommodate more knowledge and more extensive inference. The architectural mechanism is a representation that a reasoning process can apply. Its effectiveness remains dependent on which expertise was captured, the conditions under which it applies, and how it is updated. A traceable rule can expose a reason for a conclusion without demonstrating that the rule is complete or current.

Knowledge acquisition and maintenance therefore remain substantive constraints. Statistical learning offers a different way to obtain some relationships from observations; it does not eliminate decisions about labels, relevant outcomes, exceptions, or evaluation. The two approaches can be combined when explicit knowledge constrains a learned component or supplies a policy that should not be inferred from historical behavior.

C. Preference-Oriented and Collaborative Support

Preference modeling and collaboration address related but distinct requirements. Multiple-criteria decision aiding helps compare alternatives when objectives conflict and trade-offs must be represented. Preference learning investigates ways to infer or construct preference models; the fields share goals while differing in assumptions and methods [9]. This decision-formulation problem can arise for one person as well as a group.

Collaborative DSS, by contrast, address participation, communication, and shared access among decision makers [1], [2]. Networking increases the practical reach of collaboration, while interfaces and procedures determine how participants contribute and compare views. The need to exchange information is separate from agreement about how alternatives should be valued.

Greater computation can support more alternatives, sensitivity analyses, or simultaneous users. It cannot determine an acceptable trade-off or establish whose preferences have authority. A collaborative interface also cannot guarantee meaningful participation or consensus. Preference-oriented and collaborative support consequently remain cross-cutting configurations that can accompany models, warehouses, prediction, or language-based assistance.

D. Integrated Data and Analytical Support

Cross-functional decisions often depend on records distributed across operational systems with different identifiers, definitions, and update schedules. Data warehousing and online analytical processing (OLAP) address the resulting integration and workload requirements [10]. Their architectural contribution includes separating analytical work from transaction processing and providing representations suited to repeated analysis.

Storage and processing improvements permit larger histories and more demanding queries. Integration still requires cleaning, metadata, shared definitions, and maintenance responsibilities. Chaudhuri and Dayal describe the trade-off between rapid departmental data-mart deployment and the longer-term integration problems that can arise without an adequate enterprise model [10]. Thus a locally effective technical response can relocate work rather than remove it.

Data mining addresses another requirement: access to records or aggregates does not itself reveal useful patterns. Fayyad *et al.* place mining within a larger knowledge-discovery process involving selection, preparation, interpretation, and evaluation [11]. Web delivery and distributed execution are enabling mechanisms within these arrangements. They broaden access or processing capacity; they are not themselves distinct decision methodologies.

E. Predictive and Representation-Learning Support

Predictive support estimates outcomes or ranks cases from observations. Random forests provide an ensemble example, while broader machine-learning literature describes learning from experience across many tasks [12], [13]. Deep learning extends representation learning so that some features are learned from raw or less manually structured inputs [14]. These mechanisms can reduce particular forms of manual rule or feature construction, subject to suitable data and training conditions.

Capacity affects the models and datasets that can be trained and served. Architecture determines the representation and learning procedure that use those resources. The resulting predictions still require a decision rule: a forecast alone does not specify the costs of error, an acceptable intervention, or the constraints governing that intervention.

Interpretability methods provide ways to examine model predictions [15], [16]. Their explanations concern model behavior; they do not independently establish causal validity, fairness, or suitability under changing conditions. Predictive quality and decision quality must therefore be evaluated separately. An accurate demand forecast, for example, can still lead to a poor inventory choice if shortage costs or operational constraints are misrepresented. This example is illustrative, not a reported empirical result.

F. Language-Mediated, Evidence-Linked, and Tool-Mediated Support

Language models can provide a reusable interface for tasks such as expressing questions, summarizing documents, and describing alternatives. Transformer architectures and few-shot language-model results are relevant enabling developments [17], [18]; chain-of-thought prompting examines elicited intermediate reasoning on evaluated tasks [19]. These contributions concern model capabilities and should not be treated as evidence that a new generation of organizational DSS was universally established at the dates of those publications.

Retrieval-augmented generation changes access to evidence by coupling a generator with a non-parametric collection [20]. This provides a mechanism for consulting external or updated material without relying exclusively on information encoded in model parameters. Its suitability depends on retrieval relevance, document access, provenance, and whether the generated claim is supported by the retrieved passage. An attached citation is an opportunity for inspection, not a guarantee of support.

Tool-mediated support extends the interaction from generating a response to obtaining observations or executing operations. ReAct interleaves reasoning, actions, and observations; Toolformer investigates learning when and how to call tools; AutoGen explores conversational orchestration among agents and other components [21]–[23]. Self-Refine examines iterative feedback generated by a language model [24]. These mechanisms differ in control flow and evidence access, and should not be collapsed into a single claim that more agents or iterations improve decisions.

The consequences of incorrect action require particular attention. Valmeekam *et al.* report limited executable planning by the language models and tasks they evaluated [25]. That finding does not determine the capability of every later model, but it supports checking whether a plan satisfies task constraints and can be executed, rather than judging it by linguistic plausibility. Tool use can delegate calculation or retrieval to a specialized component; validation of the overall decision and its authorization remains a system-level requirement.

TABLE I
COMPARISON OF DECISION-SUPPORT CONFIGURATIONS. ROWS DESCRIBE SUPPORT PURPOSES AND ARRANGEMENTS, NOT A UNIVERSAL CHRONOLOGY OR MUTUALLY EXCLUSIVE GENERATIONS.

Configuration	Recurring requirement	Capacity contribution	Architectural or formulation response	Persistent limitation
Interactive model-based support [1], [3], [4]	Explore alternatives and assumptions	Repeated calculation at usable latency	Model–data–dialogue integration; sensitivity analysis	Model validity, calibration, usability and objective choice
Knowledge-oriented support [7], [8]	Apply domain expertise consistently	More knowledge and inference within resource limits	Explicit knowledge representation and reasoning	Acquisition, exceptions, maintenance and scope
Preference-oriented support [9]	Make trade-offs among objectives	More comparisons and sensitivity analyses	Explicit preference models and decision aiding	Legitimate values and authority to select trade-offs
Collaborative support [1], [2]	Combine distributed knowledge and participation	Connectivity, concurrent access and communication	Shared interfaces, communication and group processes	Interpretation, participation and unresolved disagreement
Integrated analytical support [10], [11]	Integrate sources and discover useful patterns	Storage, query throughput and processing scale	Warehouse/OLAP; preparation and discovery pipeline	Semantics, data quality, integration and relevance
Predictive support [12]–[14]	Estimate outcomes from observations	Feasible training and serving workloads	Learned models, ensembles and representations	Drift, explanations and mismatch with decision costs
Evidence-linked language assistance [17], [18], [20]	Express information needs and inspect external evidence	Model inference, search and indexing	Language interface coupled to retrieval	Unsupported answers, incomplete retrieval and access control
Tool-mediated workflow support [21]–[23], [25]	Obtain observations and perform linked operations	Repeated inference and tool execution	Explicit control flow among models, tools and observations	Execution validity, permissions, cost and accountability

IV. DISTINGUISHING CAPACITY EFFECTS FROM ARCHITECTURAL EFFECTS

A. Distributed Execution: A Scheduling Mechanism

MapReduce combines a programming model with an implementation that handles distribution and failures [26]. Adding machines supplies potential capacity; dividing tasks, moving data, scheduling work, and handling slow or failed workers determine how that capacity is used. The distinction matters because a larger cluster can still be constrained by coordination or a small number of unfinished tasks.

One reported experiment makes the architectural contribution concrete. In the authors' approximately one-terabyte sorting workload, completion took 891 seconds with backup tasks and 1283 seconds when they were disabled [26], Secs.5.3–5.4. The comparison concerns a scheduling mechanism in the reported cluster setting. It is not an organizational DSS outcome, and the availability of the same cluster does not imply equal resource consumption between execution strategies. The result nevertheless shows why performance cannot be attributed solely to a machine count or processor specification.

The workload also matters. Batch throughput and interactive latency are different requirements. An execution framework that processes a large offline dataset effectively may be unsuitable for a short-turnaround decision without changes in data preparation, caching, serving, or interaction design. The appropriate comparison must therefore preserve the task and latency requirement, rather than treating all faster computation as equivalent.

B. Attention-Based Models: Architecture Uses Available Hardware

The Transformer replaces recurrent sequence processing with an attention-based architecture that supports more parallelizable training in its evaluated setting [17]. The paper reports a large model trained for about 3.5 days on eight NVIDIA P100 GPUs and an English–German translation result of 28.4 BLEU on WMT2014 [17], abstract and Secs.5.2–6.1. These are reported technical conditions and outcomes, not a new experiment in this review.

The example illustrates joint dependence on architecture and resources: a different computation can expose parallelism that available accelerators can exploit. It does not isolate every contribution of architecture, training data, optimization, and hardware, nor does translation performance measure the quality of a managerial decision. For DSS, the defensible implication is that architectural changes can alter resource feasibility and interface capability; adoption and decision benefit require additional evidence.

C. External Evidence: Access Changes Without Equivalent Model Scaling

Retrieval augmentation provides a different type of change. Its motivating issues include access to knowledge, updating information, and provenance [20]. A larger generator may store more information, but coupling generation to a collection changes the route by which evidence reaches an answer. The intervention is consequently not adequately described by model size alone.

Capacity still matters for indexing, retrieval, and inference. Architecture determines whether current or restricted information can be supplied and inspected. The residual evaluation questions concern whether the right information was retrieved, whether its use was permitted, and whether the answer follows from it. More computation can improve some components without resolving these evidence and decision requirements.

V. COMPARATIVE SYNTHESIS AND DESIGN IMPLICATIONS

A. Evolution Through Combination and Constraint Shifts

Table I supports a view of DSS evolution as accumulation and recombination. The same system can integrate a warehouse, predictive model, constrained optimizer, policy rules,

and conversational interface. A new component may address a workload, access, or interaction constraint while preserving earlier components that remain appropriate. Power's classification and historical DSS accounts support this overlap [1], [2].

The most useful explanatory question is therefore which constraint is binding in a particular context. More compute is a plausible response when a suitable model and reliable inputs cannot meet throughput or latency requirements. Redesign becomes relevant when the representation cannot express the problem, evidence cannot be reached, or components cannot coordinate. Neither response determines the objectives or supplies authority to act.

This synthesis should not be read as proof that each residual limitation caused the next approach to emerge. The reviewed literature supplies stronger evidence for mechanisms than for a complete historical causal sequence. The paper's analytical contribution is to make those mechanisms comparable and expose what a proposed change leaves unresolved.

B. Diagnostic Questions for Scaling or Redesign

Four questions organize a practical diagnosis. First, is the system solving the right decision problem, with explicit objectives and constraints? Second, are the needed data and expertise available and suitable? Third, can the representation and component arrangement support the required inference, interaction, or action? Fourth, can that arrangement operate within its resource and timing requirements? These questions can be revisited as a bottleneck moves; they are not a validated decision algorithm.

Table II distinguishes candidate responses. A resource diagnosis should compare implementation performance while keeping the decision task and evaluation conditions explicit. An architectural diagnosis should identify which capability or information path changes, rather than relying on a new technology label. A formulation diagnosis should identify the objective, constraint, or participation problem that computation cannot choose on the user's behalf.

C. Evaluation Beyond Predictive Accuracy

A proposed evaluation framework follows from the preceding distinctions. Evaluate evidence quality, model or rule validity, decision quality, interaction and workflow effects, and operational performance separately. Evidence quality includes coverage and provenance. Model validity includes behavior on relevant cases and changes in operating conditions. Decision quality depends on stated objectives, costs and constraints. Workflow evaluation examines user understanding, correction and actual task completion. Operational evaluation includes latency, resources, maintenance and action failures.

The framework is an organizing proposal, not an experimentally validated instrument or a replacement for domain-specific standards. Its purpose is to prevent a narrow metric from standing in for the entire decision process. The practical and decision-theoretic grounding of DSS has long been a concern in the field [27]; selecting technically impressive benchmarks alone does not address it.

Where feasible, studies can compare a baseline, a larger resource allocation within that design, and an alternative architecture with transparent resource and human-work costs. Outcomes should be reported for the same decision task or with task differences explicitly stated. For retrieval, assess retrieval and answer support separately. For tool use, assess successful execution and constraint satisfaction alongside response quality. For collaboration and preferences, inspect participation and the legitimacy of the adopted criteria rather than treating agreement as an automatic success.

VI. Limitations and Research Directions

The review's purposive selection and uneven access limit its coverage. Historical descriptions rely on a combination of primary frameworks and later reviews; technical studies frequently concern prediction or language benchmarks rather than deployed DSS. The paper does not measure adoption, estimate how much of a transition was caused by hardware, or establish a universal ordering of approaches. The comparison matrix is an analytical organization of recurring requirements.

The review also does not provide a systematic assessment of all implementations, contemporary models, or sectors. Recent language and agent studies are examples of mechanisms, not a claim that the selected papers define the current performance frontier. References to preprints and working papers identify accessible versions rather than assuming that a separately published version was examined. Final source and venue-specific checks remain necessary for submission.

Several research questions follow. Under what task and resource conditions does scaling improve decision outcomes more than changing representations? When does additional orchestration provide enough benefit to justify its execution and maintenance costs? How can systems distinguish uncertainty about facts from disagreement about values? Longitudinal organizational studies and well-documented comparative deployments would help connect technical mechanisms to the historical and practical effects that benchmark results alone cannot establish.

VII. Conclusion

Computational capacity and architecture contribute differently to DSS development. Capacity determines which workloads can be performed at acceptable scale, latency, and cost. Architecture shapes how evidence, representations, users and actions are connected. Data, expertise and decision requirements determine whether the resulting capability is appropriate for the decision being supported.

The literature supports overlapping configurations rather than a single replacement sequence. Models, explicit knowledge, preferences, collaboration, integrated data and learned components can remain useful within the same system. Recent language, retrieval and tool mechanisms extend the combinations available, while leaving questions of validity, objectives, evidence and accountability unresolved. A useful explanation

TABLE II
PROPOSED DIAGNOSTIC DISTINCTIONS. THESE ARE SYNTHESIS-BASED DESIGN QUESTIONS, NOT EMPIRICALLY VALIDATED PRESCRIPTIONS.

Observed limitation	Question to investigate	Candidate response	Evidence required before claiming improvement
Slow execution or excessive workload cost	Is the present computation suitable and the implementation efficient?	More resources, parallelism, scheduling or serving changes	Task-matched latency, cost, throughput and output-quality comparison
Missing or inaccessible information	Does the required evidence exist, and can it be used appropriately?	Better data collection, integration, retrieval, provenance or access arrangements	Coverage, freshness, permission, relevance and support for conclusions
Inadequate representation or workflow	Can the system express constraints, coordinate components and validate actions?	Model/interface redesign, explicit control flow or specialized tools	Feasibility, error handling, constraint satisfaction and executable outcomes
Conflicting goals or unclear authority	Who defines acceptable trade-offs and authorizes action?	Preference elicitation, participation and decision-right clarification	Justified objectives, recorded assumptions and accountable approval

of DSS evolution therefore identifies the constraint addressed, the mechanism introduced, and the requirements that still need to be met. That distinction provides a more defensible basis for choosing between scaling, redesign, and changes to decision formulation.

AI ASSISTANCE DISCLOSURE

AI tools assisted text revision, source checking and document preparation. No new experiments or human-participant data collection are reported. Authorship remains unattributed in this manuscript; eventual submitting authors are responsible for source verification and the selected venue's authorship and disclosure requirements.

REFERENCES

[1] J. P. Shim, M. Warkentin, J. F. Courtney, D. J. Power, R. Sharda, and C. Carlsson, "Past, Present, and Future of Decision Support Technology," Decision Support Systems, vol. 33, pp. 111–126, 2002. [Online]. Available: https://scholarworks.uni.edu/facpub/3462/

[2] D. J. Power, "Specifying an Expanded Framework for Classifying and Describing Decision Support Systems," Communications of the Association for Information Systems, vol. 13, art. 13, pp. 158–166, 2004, doi: 10.17705/1CAIS.01313. [Online]. Available: https://aisel.aisnet.org/cais/vol13/iss1/13/

[3] J. D. C. Little, "Models and Managers: The Concept of a Decision Calculus," MIT Sloan School of Management, Working Paper 403-69, June 1969. [Online]. Available: https://archive.org/details/modelsmanagersco00litt

[4] G. A. Gorry and M. S. Scott Morton, "A Framework for Management Information Systems," MIT Sloan School of Management, Working Paper 510-71, Feb. 1971. [Online]. Available: https://archive.org/details/frameworkformana00gorr

[5] F. A. Budianto, M. Lubis, I. Y. Mukti, and S. Budianto, "The Evolution of Decision Support Systems (DSS) to Strategic AI: A Systematic Review of Architectural Shifts and Business Value," *RIGGS: Journal of Artificial Intelligence and Digital Business*, vol. 5, no. 1, pp. 8905–8916, 2026, doi: 10.31004/riggs.v5i1.6990. [Online]. Available: https://journal.ilmudata.co.id/index.php/RIGGS/article/view/6990

[6] E. J. Eze, F. M. Bello, and O. T. Akinyemi, "Decision Support Systems: Evolution, Architectures, Applications, and Future Directions," *Data Intelligence and Informatics Practice*, vol. 1, no. 1, pp. 19–31, 2026, doi: 10.53623/diip.v1i1.1264. [Online]. Available: https://doi.org/10.53623/diip.v1i1.1264

[7] E. A. Feigenbaum, "The Art of Artificial Intelligence: Themes and Case Studies of Knowledge Engineering," in Proc. IJCAI, 1977, pp. 1014–1029. [Online]. Available: https://www.ijcai.org/proceedings/1977/

[8] S.-H. Liao, "Expert System Methodologies and Applications: A Decade Review from 1995 to 2004," Expert Systems with Applications, vol. 28, pp. 93–103, 2005, doi: 10.1016/j.eswa.2004.08.003. [Online]. Available: https://doi.org/10.1016/j.eswa.2004.08.003

[9] E. Hüllermeier and R. Słowiński, "Preference Learning and Multiple Criteria Decision Aiding: Differences, Commonalities, and Synergies—Part I," 4OR, vol. 22, pp. 179–209, 2024, doi: 10.1007/s10288-023-00560-6. [Online]. Available: https://link.springer.com/article/10.1007/s10288-023-00560-6

[10] S. Chaudhuri and U. Dayal, "An Overview of Data Warehousing and OLAP Technology," ACM SIGMOD Record, vol. 26, no. 1, pp. 65–74, 1997. [Online]. Available: https://sigmodrecord.org/1997/03/15/an-overview-of-data-warehousing-and-olap-technology/

[11] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth, "From Data Mining to Knowledge Discovery in Databases," AI Magazine, vol. 17, no. 3, pp. 37–54, 1996. [Online]. Available: https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/1230

[12] L. Breiman, "Random Forests," Machine Learning, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324. [Online]. Available: https://link.springer.com/article/10.1023/A:1010933404324

[13] M. I. Jordan and T. M. Mitchell, "Machine Learning: Trends, Perspectives, and Prospects," Science, vol. 349, no. 6245, pp. 255–260, 2015, doi: 10.1126/science.aaa8415. [Online]. Available: https://doi.org/10.1126/science.aaa8415

[14] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," Nature, vol. 521, pp. 436–444, 2015, doi: 10.1038/nature14539. [Online]. Available: https://doi.org/10.1038/nature14539

[15] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You? Explaining the Predictions of Any Classifier," in Proc. KDD, 2016, doi: 10.1145/2939672.2939778, arXiv:1602.04938. [Online]. Available: https://arxiv.org/abs/1602.04938

[16] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," arXiv:1705.07874, 2017. [Online]. Available: https://arxiv.org/abs/1705.07874

[17] A. Vaswani et al., "Attention Is All You Need," in Proc. NIPS, 2017, arXiv:1706.03762. [Online]. Available: https://arxiv.org/abs/1706.03762

[18] T. B. Brown et al., "Language Models Are Few-Shot Learners," arXiv:2005.14165, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[19] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in Proc. NeurIPS, 2022, arXiv:2201.11903. [Online]. Available: https://arxiv.org/abs/2201.11903

[20] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," arXiv:2005.11401, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[21] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023, arXiv:2210.03629. [Online]. Available: https://arxiv.org/abs/2210.03629

[22] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," arXiv:2302.04761, 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[23] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[24] A. Madaan et al., "Self-Refine: Iterative Refinement with Self-Feedback," arXiv:2303.17651, 2023. [Online]. Available: https://arxiv.org/abs/2303.17651

[25] K. Valmeekam, S. Sreedharan, M. Marquez, A. Olmo, and S. Kambhampati, "On the Planning Abilities of Large Language Models (A Critical

Investigation with a Proposed Benchmark)," arXiv:2302.06706, 2023. [Online]. Available: https://arxiv.org/abs/2302.06706

[26] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," in Proc. OSDI, 2004, pp. 137–150. [Online]. Available: https://research.google/pubs/mapreduce-simplified-data-processing-on-large-clusters/

[27] D. Arnott and G. Pervan, "Eight Key Issues for the Decision Support Systems Discipline," Decision Support Systems, vol. 44, pp. 657–672, 2008, doi: 10.1016/j.dss.2007.09.003. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167923607001698